

Large Language Model driven Policy Exploration for Recommender Systems

Jie Wang
j.wang.9@research.gla.ac.uk
University of Glasgow
Glasgow, UK

Ioannis Arapakis
ioannis.arapakis@telefonica.com
Telefonica Research
Barcelona, Spain

Alexandros Karatzoglou
alexandros.karatzoglou@gmail.com
Amazon
Barcelona, Spain

Joemon M. Jose
joemon.jose@glasgow.ac.uk
University of Glasgow
Glasgow, UK

Abstract

Recent advancements in Recommender Systems (RS) have incorporated Reinforcement Learning (RL), framing the recommendation process as a Markov Decision Process (MDP). However, offline RL policies trained on static historical data are vulnerable to distribution shift when deployed in dynamic online environments. Additionally, excessive focus on exploiting short-term relevant items can hinder exploration, leading to sub-optimal recommendations and negatively impacting long-term user gains. Online RL-based RS also face challenges in being deployed in production, due to the risks of exposing users to untrained or unstable policies. Large Language Models (LLMs) offer a promising solution to mimic user objectives and preferences through language, making them suitable for pre-training policies offline to enhance the initial recommendations in online settings. Effectively managing distribution shift and balancing exploration are crucial for improving RL-based recommenders, especially when leveraging LLM-based pre-training.

To address these challenges, we propose an Interaction-Augmented Learned Policy (iALP) that utilizes user preferences distilled from an LLM. Our approach involves prompting the LLM with user states to extract item preferences, learning rewards based on feedback, and updating the RL policy using an actor-critic framework. Furthermore, to deploy iALP in an online scenario, we introduce an adaptive variant, A-iALP, that implements a simple fine-tuning strategy (A-iALP_{ft}), and a second adaptive approach (A-iALP_{ap}) designed to mitigate issues with compromised policies and limited exploration. Experiments across three simulated environments demonstrate that A-iALP introduces substantial performance improvements.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Online Reinforcement Learning for Recommendation; Long-term Gains; Exploration Strategies

ACM Reference Format:

Jie Wang, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M. Jose. 2025. Large Language Model driven Policy Exploration for Recommender Systems. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25)*, March 10–14, 2025, Hannover, Germany. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/XXXXXX.XXXXXX>

1 Introduction

Modelling user states through interacted items to generate future recommendations has become a common paradigm for many recommender systems (RS), particularly in session-based or sequential models [11, 15, 38, 48]. Traditionally, these methods implement offline training on historical user log data, using self-supervised techniques. To address long-term user benefits, recent research reformulated this process as a Markov Decision Process (MDP) [32, 44, 45], introducing Reinforcement Learning (RL) to solve the associated decision-making challenges.

However, efforts to develop offline RL policies using historical user data [32, 44, 45] have been shown to be vulnerable to distribution drift, when deployed online [5, 18]. This can lead to erroneous actions in out-of-distribution states due to the absence of real-time user feedback from interactions with the environment. Furthermore, offline RL-based systems often suffer from insufficient exploration of the data space, resulting in sub-optimal model performance. These limitations are particularly pronounced in cold-start scenarios and during initial stages when training data is sparse, causing policies to perform poorly and fail to provide satisfactory recommendations. To address these challenges and improve long-term gains, effective methods that can thoroughly explore the data space while adapting to real-time user interactions are needed.

To this end, researchers have explored online RL methods for recommendation that learn from real-time user feedback, as illustrated in Figure 1. For example, DQN [26], A2C [16], and DDPG [21] have been used to optimize cumulative rewards, aiming to enhance long-term user engagement. However, these methods typically require extensive training iterations before achieving optimal performance. Moreover, they overlook a crucial issue that can lead to user churn: if users receive unsatisfactory recommendations during

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
WSDM '25, March 10–14, 2025, Hannover, Germany.
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1329-3/25/03
<https://doi.org/10.1145/XXXXXX.XXXXXX>

their initial interactions with a system, they are likely to abandon it rather than stay and provide feedback for future improvements. This challenge is one of the main reasons [44] why online RL has not gained widespread adoption in RS.

LLMs with knowledge transfer capabilities have recently gained significant attention in RS [3, 10, 49]. In offline settings, LLMs have demonstrated potential as zero-shot [1] or pre-trained [7] RS, achieved through fine-tuning on user data. Moreover, LLMs have been shown to capture user objectives, learnt from expert demonstrations or preferences, through their intuitive use of language [40]. This suggests the potential to leverage LLMs to interpret user preferences and generate appropriate actions across various states. Such feedback can be utilised to pre-train a policy offline, enabling desirable initial behaviours for online recommendations.

To further enhance the performance of an RL-based RS, we introduce an Interaction-Augmented Learned Policy (iALP). We initially train the policy by exclusively interacting with an LLM. We first derive user preferences from an offline LLM, which are subsequently employed for online user interactions. To derive preferences for various items, we prompt the LLM with task-specific instructions to select potential actions. The LLM assigns rewards (1 or 0) for candidate items based on the prompted preferences. This process generates an interactive offline dataset, entirely generated by the LLM, which is used to continuously update the RL policy using an actor-critic framework [16].

For transitioning the policy to online, we propose an adaptive Interaction-Augmented Learned Policy (A-iALP) variant. Specifically, we introduce the direct fine-tuning strategy A-iALP_{ft} to initialise and update the online agent with real feedback from the user environment. A-iALP_{ft} can be deployed as an on-line agent to effectively address policy drift issues and limited exploration, two challenges with off-line RL. In addition, we propose a second adaptive approach, A-iALT_{ap}, which combines a frozen pretrained agent with an online policy to allow further learning. Specifically, the useful behaviours of iAPT are retained to generate desirable sequences in the initial training stage, and in the later stage the actions from the online policy are mainly used to interact with the environment. Furthermore, we validate the performance of A-iALP under different exploration strategies against established baselines. Experimental results on three simulated environments demonstrate the superior performance A-iAPT in generating desirable initial and stable episodes. Our contributions can be summarised as follows:

- We propose to distil user preferences on items using LLMs and use these preferences to train an interaction-Augmented Learned Policy (iALP).
- We introduce two adaptive methods for iALP-to-Online (A-iALP) RL-based recommendation, leading to faster and stable online policy convergence, to enhance long-term rewards (return) and alleviate gain loss for users at early steps.
- We conduct experiments on three recommendation datasets. Experimental results show significant improvements in all metrics.

2 Related Work

2.1 RL for Recommendation

Recommenders, e.g., session-based or sequential recommendation methods [11, 15, 38, 48], are often trained offline on historic user

data, in a self-supervised manner. To target long-term gains for users, recent research has reformulated the process as a Markov Decision Process (MDP), introducing RL to solve the associated decision-making challenges [27, 42, 51]. Specifically, Liu et al. [24] extended DDPG to session-based recommendation, while Xin et al. [44] introduced self-supervised RL for Sequential Q-Network (SQN) and Soft Actor-Critic (SAC) to improve the accuracy of the baselines with respect to user clicks and purchases. Xin et al. [45] further enhanced the frameworks with SNQN and SA2C by using a sampling strategy to integrate negative feedback. Another approach [46] involves guiding training by modelling desirable cumulative rewards rather than focusing on expected returns at each timestamp. Furthermore, Ren et al. [32] increased the original states to improve SQN with contrastive learning. Despite these advancements, the aforementioned methods are primarily trained offline on historical log data, resulting to sub-optimal policies due to potential distribution drift when deployed online. Furthermore, focusing on recommending the most relevant items based on historical sessions can introduce biases that negatively impact user experience by neglecting long-term engagement. To optimise long-term gains for users, some recent work explored online RL-based recommendation and reward-related metrics [47]. However, this approach requires extensive training steps to achieve satisfactory results. In our paper, we primarily explore how to accelerate the enhancement of users' long-term benefits while mitigating the loss of user satisfaction in the initial stage.

2.2 LLMs for Recommendation

LLMs [23, 25, 30, 39, 50] pre-trained on massive natural language datasets with continuously enhanced transfer capabilities have gained attention in the field of RS [2, 4, 6, 22]. Existing offline adaptations of LLMs for recommendation tasks involve primarily training the LLM to be a new recommender through pre-training [7, 12, 41], fine-tuning [25], prompt-based tuning [1, 23, 29] and item representation [31], data augmentation [40] etc. For online RL-based recommendation, [37] adapts LLM directly as an online agent in aspects of actor/critic/planner to improve long-term engagement of users. However, these methods of employing LLMs as recommenders come with significant pre-training costs or face difficulties in preserving LLM signals. Recent work by Kwon et al. [19] demonstrated that the use of LLMs as reward models can significantly outperform traditional learnt rewards in the context of RL applications for games. Inspired by this, we propose the utilisation of LLMs for specifying user objectives and preferences through natural language, making them suitable for pre-training policy offline to boot the initial recommendations online. In addition, such an approach facilitates rigorous exploration of data spaces to create effective models.

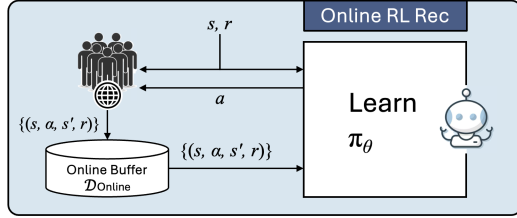
3 Preliminary

3.1 RL-based Long-term Recommendation

Online recommender systems aim to retrieve items that can enhance the user experience and provide long-term engagement. RL-based RS for long-term user engagement/satisfaction is based on the principles of a Markov Decision Process (MDP) [44]. The agent

Table 1: Prompt template to generate user preference from LLM.

	Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.
Instruction:	You are a user in a {scenario} platform now. The {item} is in the form of {attribute1; attribute2; ... }. Given a user's {behavior} history of {item}, and candidate {item} labelled by lowercase letter to be decided to recommend to the user, identify which {item} the user will mostly prefer to at next timestamp. Please judge by the user's preference on {attribute1; attribute2; ... }; if you think that none of the candidates will be selected by the user, please answer "None"
Input:	History: {interacted items}. Which one the user will mostly like at next timestamp in the following candidates? a. {item1} b. {item2} ... j. {item10} k. None
Response:	By analysing the user's preference, the user will select {label from a-k}

**Figure 1: Process of online RL for recommendation.**

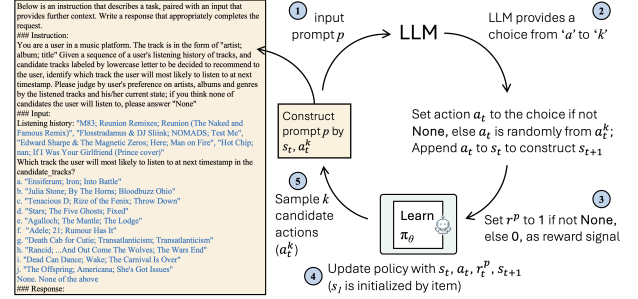
(recommender) interacts with the environment (users), taking actions (recommending items) based on the state of the environment, which is represented by the interacted items of the user. The agent is then updated by the feedback (reward) from the user. Continually, the environment generates a new state for the agent as users respond to the actions. Figure 1 shows the process, represented by the tuple $(S, \mathcal{A}, \mathcal{P}, r, \gamma)$:

- State space $\mathcal{S}$ represents the user's state. $s_t \in \mathcal{S}$ is the state at timestamp t , which is usually generated by mapping the sequence of interacted items (user history) into a sequential model/state encoder.
- Action space $\mathcal{A}$. The discrete action set is comprised of the candidate items. Taking action means recommending items. In the online setting, the action $a_t \in \mathcal{A}$ at timestamp t will be the interacted item in the next timestamp to construct the next state.
- Reward function r returns immediate reward r_t given state s_t and the action a_t taken by the agent at timestamp t , which reflects the user's feedback on current recommendation, e.g., likes, dislikes.
- State transition function $\mathcal{P}$ describes the next state s_{t+1} from the environment given the current state s_t and observed action a_t .
- Discount factor γ to the future rewards, where $\gamma \in [0, 1]$.

The RL-based recommendation aims to learn a target policy $\pi_\psi(a|s)$ that maps the state $s \in \mathcal{S}$ to an action distribution $a \in \mathcal{A}$ by maximizing the expected cumulative rewards (return) to realize long-term user engagement, where θ denotes the parameters:

$$\max_{\pi_\theta} \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)], \text{ where } R(\tau) = \sum_{t=0}^{|\tau|} \gamma^t r(s_t, a_t), \quad (1)$$

where τ denotes the trajectory of (s_t, a_t, s_{t+1}) .

**Figure 2: Generate user preference, e.g., action and reward, for offline pre-training.**

4 Method

We first introduce the interaction-Augmented Learned Policy (iALP), which is based on item preferences extracted from a Large Language Model. Following that, we outline strategies for adapting iALP (A-iALP) to an online RL-based recommendation system.

4.1 Recommendation Policy trained on Preferences from LLM

4.1.1 Preference Distillation. The principle of obtaining user preferences from an LLM involves using it as judge to evaluate which items the user is likely to prefer or dislike, based on their interaction with the LLM. We design a preference prompt $p(s, a^k)$ to obtain this knowledge. Table 1 presents the prompt template, which is guided by the description of the recommendation task for a specific scenario (e.g. scenario = track / video). The template uses the textual attributes of items (e.g., attribute1=title, attribute2=brand) to define the recommendation context. As illustrated in Figure 2, the prompt contains the state s and a list of k ($k=10$ in the prompt example) actions a^k , which are fed to an LLM to provide a response, a choice a from 'a' to 'k'. If the output is not None the corresponding action is selected for the user, otherwise the action is randomly selected from a^k . We can see this procedure as an LLM-based sub-policy to select action:

$$a \sim LLM[p(s, a^k)] \quad (2)$$

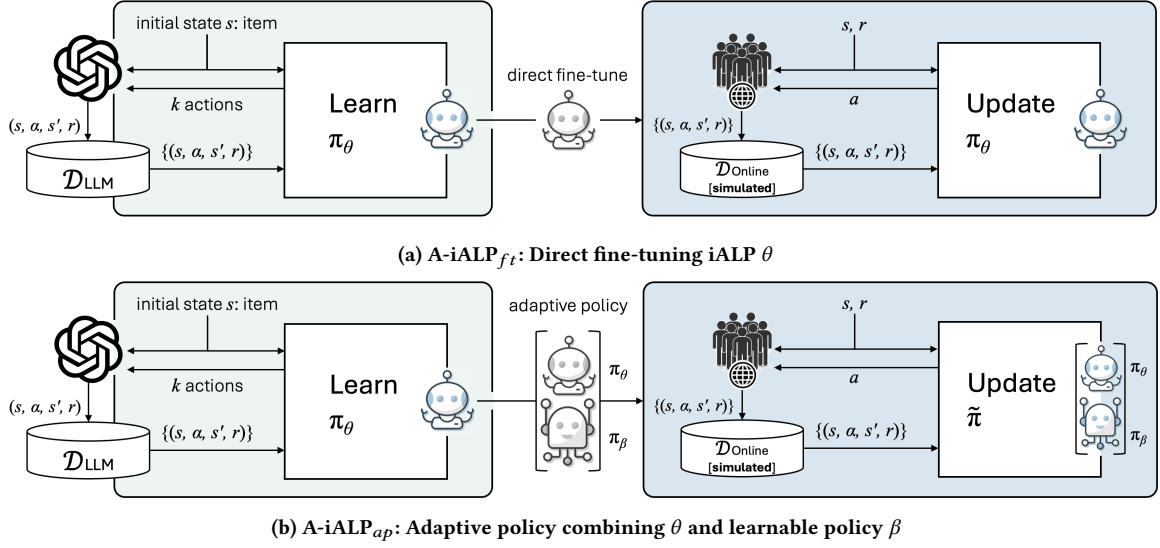


Figure 3: Illustration of different adaptation schemes of iALP-to-online. $\mathcal{D}_{online}$ consists of generated actions a from policy π_θ and corresponding rewards from the reward model.

Next, the reward r^P for the action is designed as:

$$r^P = \begin{cases} 1.0 & \text{if the action is selected} \\ 0.0 & \text{if the response is None} \end{cases} \quad (3)$$

4.1.2 Policy Pre-training. We utilize these responses as reward signals and training data to pre-train the recommendation policy offline. As the Figure 2 shows, once feedback from the LLM on actions is received, we train the interaction-Augmented Learned Policy (iALP) using the actor-critic (A2C) architecture [16]. This is combined with SASRec [15] as a state encoder $G(\cdot)$, a widely adopted approach in RL-based RS. More specifically, given the interacted items $x_{1:t}$, the state at timestamp t is:

$$s_t = G(x_{1:t}), \quad (4)$$

where $x_{1:t} = \{x_1, \dots, x_t\}$ denote the interacted sequence of items, i.e., previous episode. I denote the set of items in the system, $x_i \in I (0 < i \leq t)$ is the index of the interacted item ordered by timestamp. s_t is then used for the actor and critic networks:

$$a_t \sim \pi_\theta(\cdot | s_t), \quad (5)$$

The implementation of π_θ contains two steps. First, *Actor* network maps the state s_t into action distribution, and a list of k actions a_t^k is sampled (random or top- k actions), i.e., $a_t^k \sim \text{Sample}(\text{Actor}[s_t])$. Second, the preference distillation procedure in section 4.1.1 is executed to select action a_t and generate reward r_t^P , i.e. $a_t \sim \text{LLM}[p(s_t, a_t^k)]$. The LLM can be seen as a helper policy to select the best action for the recommendation agent that finishes decision-making. Accordingly, the actor loss is formulated as:

$$\mathcal{L}_A = -\log \pi_\theta(a_t | s_t), \quad (6)$$

Regarding the Q-learning network as critic network, we compute the Q-value as follows:

$$Q(s_t, a_t) = \text{Critic}[s_t], \quad (7)$$

The one-step time difference (TD) Q-learning loss as critic loss is defined as:

$$\mathcal{L}_Q = (r^P(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))^2, \quad (8)$$

where a_t is appended to $x_{1:t}$ to generate next state s_{t+1} by $G(x_{1:t+1})$. The advantage Q-value is calculated as:

$$A(s_t, a_t) = r^P(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t), \quad (9)$$

The actor loss $\mathcal{L}_A$ is finally formulated as $\mathcal{L}_A \leftarrow \mathcal{L}_A \cdot A(s_t, a_t)$. In summary, the corresponding loss function for pre-training based on feedback from LLM is formulated as follows:

$$\mathcal{L} = \mathcal{L}_A + \mathcal{L}_Q \quad (10)$$

4.2 Simulated Online Learning

We propose two strategies to adapt iALP (A-iALP) to online recommendation for better long-term gains and alleviation of reward loss in the initial steps. Note that while in the previous phase we used the LLM as the reward and action model, here we do not utilise any LLM model.

4.2.1 Online Environment Simulation. To provide immediate feedback and update online recommendation policies, we need to build RL environments. Due to the inaccessibility and high cost of real recommendation platforms, we use a simulated environment, following the approach of Liu et al. [24] and Shi et al. [37]. This simulated environment returns rewards based on state and action and can be easily constructed using public datasets. We employ two different simulated environments tailored to specific experimental settings, generating states based on user history and providing rewards for the recommended items (see Section 5.1 for details).

We perform experiments on three user environments from different recommendation scenarios: **LFM** [34], **Industry** [28] and

Coat[35]. The LFM is a music streaming platform Last.fm¹ with textual content of title, album, and artist for each track (item). The Industry is the Industrial and Scientific category of Amazon review² with textual description of title, category and brand for each product (item). We simulate an online environment for each scenario using a publicly available sequential interaction dataset, which is summarized in Table 2.

Reward Model: For the Coat dataset, we follow the setting in [37], using Matrix Factorization (DeepFM)[9] to establish a user model, which acts as the reward model to simulate user feedback in training and testing environments. For the LFM and Industry datasets, we construct the reward model based on a sequential recommender method [24], which generates scores/rewards based on a user’s state consists of sequential items and the action. The Coat simulator focuses on user behaviors toward items, with the user as the initial state, while the latter two emphasizes the relationship between the recommended item and the user’s interaction history, using an item as the initial state.

4.2.2 iALP-to-Online Fine-tuning. Although online recommendation benefits from real user feedback for performance improvement, it is less sample efficient and experiences low returns at the beginning of training. In contrast, the pre-training RL policy is sample-efficient since no online interactions are required. Therefore, instead of treating online recommendation as an individual topic, we directly use the policy π_θ learned from LLM to generate the online policy and, in particular, we sample the action from policy π_θ :

$$a_t \sim \pi_\theta(\cdot|s_t) = \text{Actor}[s_t], \quad (11)$$

The use of the pre-trained policy π_θ essentially means that we deploy the pre-trained actor-critic network with the help of the simulator that provides the corresponding rewards r . The policy is then is fine-tuned based on the simulated real-time user reward r (feedback), we call this method A-iALP_{ft}. The online actor loss and critic loss are as follows:

$$\mathcal{L}_Q^{on} = (r(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))^2, \quad (12)$$

$$\mathcal{L}_A^{on} = -\log \pi_\theta(a_t|s_t), \quad (13)$$

The key difference of A-iALP_{ft} is that the same algorithm is further improved for online learning using the simulated environment reward r to replace the designed signal r_p in Eq. 8. The process is also illustrated in Figure 3a, which directly uses online feedback to continue to optimize π_θ .

4.2.3 Adaptive Policy. Although direct fine-tuning is straightforward, it presents several challenges. For example, the pre-trained policy might be compromised or even deteriorate during the initial phase of online training, particularly due to distribution shifts between LLM-generated preferences and actual online user behavior [20]. Another concern is that exploration [33] may be overlooked as the iALP often prioritizes non-preferred actions, limiting the system’s ability to discover newer and potentially better recommendations.

Algorithm 1 Training iALP Phase

```

1: Require: Item set  $I$  (action space  $\mathcal{A}$ ), LLM
2: Initialize replay buffer  $\mathcal{D}_{LLM}$  by initial user sequence  $x_1 \in I$ 
3: Initialize parameters of the agent  $\theta$ : actor network Actor, critic network Critic, and the state encoder  $G$ 
4: for each iteration do
5:   Obtain initial user sequence  $x_1$  from  $\mathcal{D}_{LLM}$ 
6:   for step  $t = 1, \dots, T$  do
7:     Generate state  $s_t$  by  $G(x_{1:t})$ 
8:     Execute Actor $[s_t]$  to produce action distribution
9:     Sample a list of  $k$  actions  $a_t^k$ 
10:    Construct prompt  $p(s_t, a_t^k)$  based on template
11:    Act  $LLM(p(s_t, a_t^k))$  to generate  $a_t$  (i.e.,  $x_{t+1}$ ),  $r_t$ 
12:    Store transition  $(x_{1:t}, a_t, r_t, x_{1:t+1})$  in  $\mathcal{D}_{LLM}$ 
13:    Sample minibatch  $b$  from  $\mathcal{D}_{LLM}$ 
14:    With  $b$ , calculate  $\mathcal{L}_Q$  and  $\mathcal{L}_A$ 
15:    Update  $\theta$ 
16:   end for
17: end for
18: Return Agent  $\theta$ :  $G$ , Actor, and Critic

```

To solve the above issues, we propose an alternative scheme A-iALP_{ap} (illustrated in Figure 3b) that combines the existing algorithm with the online policy to allow further learning. Given a policy π_θ obtained from pre-training phase, instead of directly fine-tuning the parameters, we freeze π_θ and propose another learnable policy π_β . Both are added into a policy set $\tilde{\pi} = [\pi_\theta, \pi_\beta]$, which is responsible for further performance improvement during online training. The final policy $\tilde{\pi}$ can be represented as follows:

$$a_t \sim \tilde{\pi}(\cdot|s_t) = (1 - \alpha)\pi_\theta(\cdot|s_t) + \alpha\pi_\beta(\cdot|s_t), \quad (14)$$

$$\mathcal{L}_A^{on} = -\log \tilde{\pi}(a_t|s_t) \quad (15)$$

where α is the weight of taking actions from the learnable policy π_β , $1 - \alpha$ indicates the probability of using the pre-trained policy π_θ . As the training steps increase, α increases to 1. In practice, the initial value of α depends on the specific recommendation scenario. It is intuitive to understand that the pre-trained policy π_θ mainly determines actions in the early stages of training, and as the new policy π_β learns more and more preferences, it will take over the decision-making process at the end of the fine-tuning process.

4.3 Discussion

Algorithms 1 and 2 illustrate the iALP pre-training and iALP-to-Online procedures, respectively. Note that the critic and state encoder networks of A-iALP_{ap} are updated the same way as A-iALP_{ft}. The total loss of online recommendations is $\mathcal{L}^{on} = \mathcal{L}_A^{on} + \mathcal{L}_Q^{on}$. We can retain the useful behaviours of iALP to generate desirable sequences in the initial online training stage. The adaptive scheme enables learning new abilities by interact with the user environment.

¹<http://www.last.fm/api>

²<https://nijianmo.github.io/amazon>

Algorithm 2 A-iALP: iALP-to-Online Phase

```

1: Require: iALP  $\theta$ , learnable agent  $\beta$ , scheme, weight  $\alpha$ 
2: Initialize empty online replay buffer  $\mathcal{D}_{\text{Online}}$ 
3: Initialize parameters of the agent  $\beta$  with  $\theta$ .
4: for each iteration do
5:   Obtain initial state  $s_0$  from environment
6:   for step  $t = 1, \dots, T$  do
7:     if scheme is A-iALP $_{ft}$  then
8:       Act  $a_t \sim \pi_\theta(\cdot|s_t)$ 
9:     else if scheme is A-iALP $_{ap}$  then
10:      Act  $a_t \sim (1 - \alpha)\pi_\theta(\cdot|s_t) + \alpha\pi_\beta(\cdot|s_t)$ 
11:     end if
12:     Obtain  $r_t, s_{t+1}$  from environment
13:     Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{D}_{\text{Online}}$ 
14:     Sample minibatch  $b$  from  $\mathcal{D}_{\text{Online}}$ 
15:     With  $b$ , calculate  $\mathcal{L}_Q^{on}$  and  $\mathcal{L}_A^{on}$ 
16:     Update  $\beta$ 
17:   end for
18: end for

```

Table 2: Dataset statistics.

Dataset	#item	#seq./user	#inter.	Item attribute
LFM	18,297	11,073	146,255	title; album; artist
Industry	5,814	10,935	71,872	title; category; brand
Coat	290	300	11,600	jacket colour; type

5 Experimental Setup

5.1 Research Questions

In this section, we describe the experimental setup used to validate our proposed A-iALP approach. We aim to answer the following research questions:

RQ1: How does iALP compare to traditional purely online RL-based recommendation methods in the initial stages of online recommendation?

RQ2: How effective is A-iALP at achieving long-term effectiveness in online recommendations?

RQ3: How does A-iALP compare to directly incorporating preference of LLMs in online training?

RQ4: How does A-iALP perform under various exploration-exploitation strategy?

Table 2 summarises the three data sets (section 4.2.1) used for the experiments. The datasets are partitioned to simulate online training (80% dataset) and testing (20% dataset) environments.

5.2 Baselines

We compare the proposed A-iALP with several online RL methods for online recommendation.

- DQN [26]: A Q-learning based off-policy method learning from data produced by a policy different from the one currently being optimized.
- PG [43]: An on-policy gradient-based method that utilizes data generated by their current policy.

- A2C [16]: An on-policy gradient-based method with the actor-critic framework, which is the same as our proposed iALP in the pre-training stage.
- iALP: The pre-trained agent to take advantage of the preferences derived from LLM and optimised based on A2C. This model serves as a baseline, providing recommendations without further exploration or updates.

The two adaptation strategies of our A-iALP are described as A-iALP $_{ft}$ and A-iALP $_{ap}$, respectively, for clarity.

5.3 Metrics

We apply three metrics: Return, Length and Average Reward [36, 47] to evaluate the RL-based RSs. Return ($R = \sum_t r_t$) measures the cumulative rewards (return) of the recommended episode/sequence, which is applied in the RL methods [36] to evaluate long-term gains. The higher Return denotes better performance. In addition, the length (Len) of the sequence and the mean reward ($R_{avg} = R/Len$) of the actions are also used as a reference [37, 47] to analyse user engagement and performance of the immediate recommendation of the policy. To evaluate in early training steps, we incorporate $R@e$, $Len@e$ and $R_{avg}@e$, where $e \in \{0, 1, 2\}$, to show the results in epoch e .

5.4 Implementation Details

For the training stage of iALP, to generate the correct label format, we first tune the LLM with the commonly used LORA in the PEFT [13] strategy to learn the template prompt via 1000 randomly sampled interactions. The LLM we use is Mistral 7B [14]. iALP is an A2C framework trained with generated preferences (action, reward) from LLM at every step for 100 epochs. In the online stage, we use the same architecture for A-iALP. The difference in A-iALP $_{ap}$ is that it freezes the iALP policy network and initialises randomly another learnable policy with the same network. For online learning, all RL methods are trained with 50k steps and evaluated in the same test environment, and the most recent 20 episodes are used to generate the learning curve results.

6 Experimental Results

In this section, we present the experimental results to answer the research questions described in Section 5.1.

6.1 Initial Performance Comparison (RQ1)

Table 3 shows the performance of directly applying iALP to recommend items without learning from online environment, i.e., $e=0$. We see that iALP significantly outperforms traditional on-line RL methods that are randomly initialised in the context of cold-start online recommendation. The results demonstrate that iALP, which is trained on preferences distilled from LLMs, can provide a relatively desirable item sequence in terms of cumulative rewards and sequence length of the recommended items. Specifically, iALP manages to alleviate the common issue of poor quality recommendations during the initial phase of deployment, which is a typical problem for traditional RL methods trained from scratch. This advantage not only improves the user experience from the outset, but also accelerates the process of online updates and learning. By leveraging the pre-trained knowledge, iALP is able to bypass the

initial random exploration phase that purely online RL methods undergo, which often results in suboptimal recommendations and user dissatisfaction. Instead, iALP starts with a more informed and refined strategy, leading to higher initial user engagement and satisfaction. We also see in Tables 4 and 5 that, after incorporating a small amount of user feedback (training 1 or 2 epochs), A-iALP maintains greater long-term returns compared to general methods. This demonstrates that A-iALP leverages effectively user feedback to enhance performance. This improved starting point facilitates gathering more relevant early feedback, enabling faster and more effective online learning and adaptation.

Table 3: The initial performance, i.e., epoch=0, on LFM and Industry environments for online RL-based recommendation. Boldface denotes the highest score.

Method	LFM			Industry		
	$R@0$	$Len@0$	$R_{avg}@0$	$R@0$	$Len@0$	$R_{avg}@0$
DQN	1.53	4.52	0.34	1.12	5.21	0.21
PG	1.73	4.73	0.37	1.06	5.15	0.22
A2C	1.96	5.15	0.38	2.43	5.92	0.41
iALP	5.11	6.21	0.82	6.35	5.94	1.06

Table 4: Performance of online RL-based recommendation on LFM and Industry environments while epoch=1. Boldface denotes the highest score.

Method	LFM			Industry		
	$R@1$	$Len@1$	$R_{avg}@1$	$R@1$	$Len@1$	$R_{avg}@1$
DQN	5.04	6.01	0.84	3.84	6.51	0.59
PG	4.85	6.32	0.77	3.32	5.96	0.56
A2C	6.12	6.84	0.89	7.32	6.41	1.14
A-iALP _{ft}	8.83	8.01	1.11	9.28	8.62	1.07

Table 5: Performance of online RL-based recommendation on LFM and Industry environments while epoch=2. Boldface denotes the highest score.

Method	LFM			Industry		
	$R@2$	$Len@2$	$R_{avg}@2$	$R@2$	$Len@2$	$R_{avg}@2$
DQN	8.32	7.43	1.12	16.3	8.32	1.96
PG	8.23	7.23	1.14	14.5	8.43	1.72
A2C	8.39	8.04	1.04	17.2	8.36	2.06
A-iALP _{ft}	14.13	8.52	1.66	22.1	9.43	2.34

6.2 Long-term Performance Comparison (RQ2)

Figures 6 and Figure 7 illustrate the training curves between baseline and A-iALP w.r.t. returns, the sequence length on the LFM and Industry, respectively. The performance of all methods are

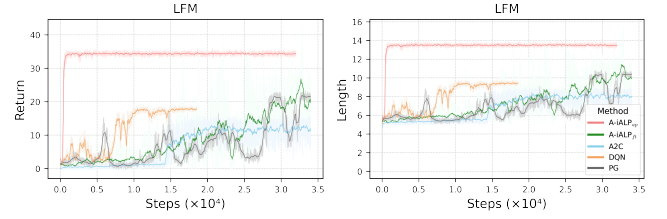


Figure 6: Learning curves of return (left) and length (right) between baselines and A-iALP on LFM environment.

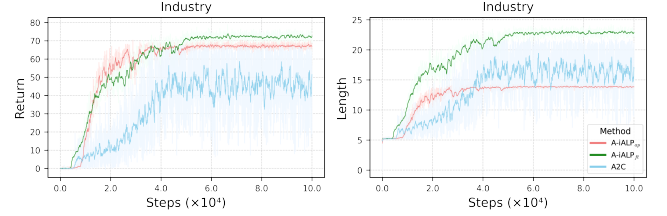


Figure 7: Learning curves of return (left) and length (right) between several baselines and A-iALP on Industry environment.

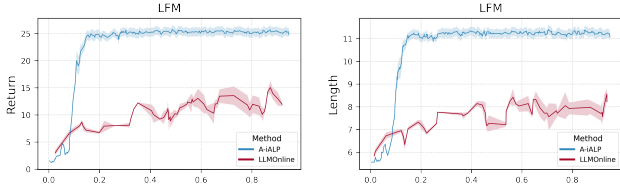
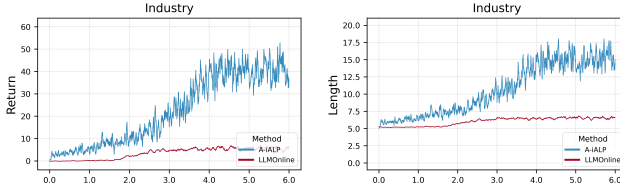
presented on the LFM, while Industry focusses on the top three methods. We can see that in both LFM and Industry A-iALP_{ap} converges quickly and demonstrates stability, which achieves best return at around 1,000 steps and 20,000 step, respectively. In Industry, A-iALP_{ft} performs second best on Industry before 40,000 environmental steps, while A-iALP_{ap} takes the first position as steps continue. In LFM, A-iALP_{ft} increases stably though with slightly lower performance at the beginning. The performance in the test environment from is illustrated in Table 6. We can observe that A-iALP in general outperforms the training from scratch methods (DQN, PG and A2C) in return and length. In the metric of length, there is no obvious differences on all methods, which might be attributed to the phenomenon that the length of recommended sequence might not reflect the user’s long-term satisfaction. Specifically, A-iALP_{ft} shows large improvements over iALP, implying the benefits brought about by additional online training over pure offline training based on LLM interactions. The proposed A-iALP_{ap} outperforms baselines method overall. It shows stable improvements starting from a desirable performance brought about by the pre-training procedure.

6.3 Utilisation of LLM (RQ3)

Here we compare A-iALP with LLMOnline, which directly uses LLM as a sub-agent in an online environment for taking actions. The goal is to verify the effectiveness of our iALP-to-Online recommendation procedure. LLMOnline is similar to pre-training iALP, but differs in that the agent is directly updated by user feedback in the online RL-based recommendation, rather than by a predefined reward function. The results on LFM (Figure 8) and Industry (Figure 9) indicate that, in the initial stages, LLMOnline performs similarly to A-iALP. However, as the environment steps increase, A-iALP demonstrates superior performance. This can be attributed to the

Table 6: Performance comparison of online RL methods for online recommendation. Boldface denotes the highest score, and the second-best results are underlined.

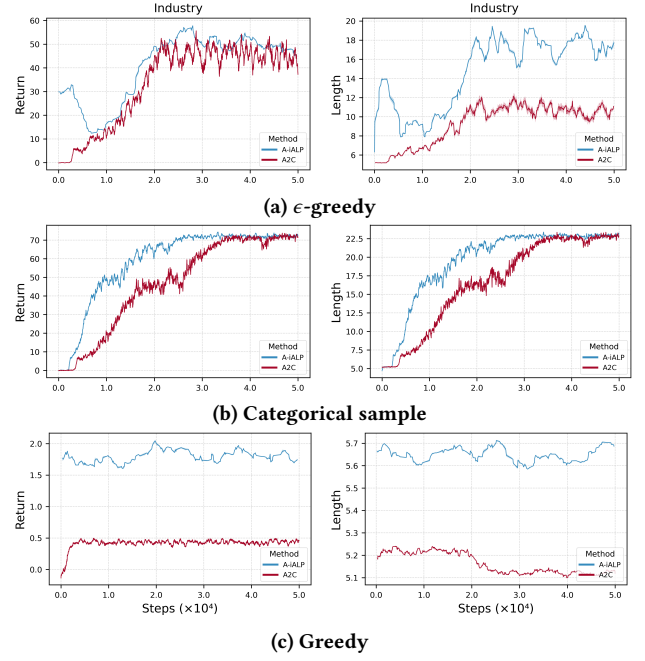
		DQN	PG	A2C	iALP	A-iALP _{ft}	A-iALP _{ap}
LFM	<i>R</i>	28.8	29.7	28.1	11.2	31.5	33.1
	<i>Len</i>	9.53	8.46	9.29	7.21	10.1	10.2
	<i>R_{avg}</i>	3.02	3.51	1.55	3.21	3.13	3.22
Industry	<i>R</i>	40.3	43.3	46.3	25.3	52.4	51.8
	<i>Len</i>	11.3	10.4	10.8	10.7	11.5	11.3
	<i>R_{avg}</i>	3.57	4.16	4.28	2.36	4.55	4.58
Coat	<i>R</i>	54.3	79.3	81.7	31.2	83.2	84.4
	<i>Len</i>	22.1	29.8	29.9	12.9	29.6	29.7
	<i>R_{avg}</i>	2.47	2.66	2.73	2.42	2.81	2.84

**Figure 8: Comparison of return (left) and length (right) between using LLM online and A-iALP method on LFM.****Figure 9: Comparison of return (left) and length (right) between using LLM online and A-iALP method on Industry.**

capacity of iALP to enable an effective exploration during the initial stage, while A-iALP refines its recommendations through continuous interaction with real-time simulated user feedback. In contrast, LLMOnline may be facing limitations as training progresses, with the model’s reliance on LLM choices potentially diminishing its ability to adapt to preferences directly from the environment. Another key consideration is the time cost of online adaptation with LLMs. LLMOnline requires significant additional time for processing and generating recommendations through prompting, which can be less efficient in an online setting. In contrast, A-iALP maintains a time cost comparable to traditional RL methods, thanks to its independent pre-training phase.

6.4 Effect of Exploration Strategy (RQ4)

Finally, we examine the impact of exploration-exploitation [17] strategies on A-iALP, including: (a) ϵ -greedy [8], which explores by a random action with probability ϵ and the greedy action with probability $1 - \epsilon$; (b) categorical sampling [24], which samples actions based on the probability distribution; (c) greedy selection, which

**Figure 10: Performance comparison between A-iALP and the baseline under various exploration strategies on Industry.**

is a baseline that exploits the action with the highest estimation without exploration and exploitation balance. Using LFM as a reference (Figure 11), A-iALP consistently outperforms the baseline A2C method across all exploration strategies. This demonstrates the effectiveness of integrating preferences distilled from LLMs at the pre-training phase, which provides a robust foundation for more efficient exploration. Specifically, with the ϵ -greedy method, A-iALP shows slightly better performance than A2C in the early training stages, with A-iALP achieving stable growth later, while A2C experiences significant fluctuations. In categorical sampling, both show stable growth, but A-iALP converges faster. Notably, under the greedy exploitation method, both maintain almost constant performance, but A-iALP’s return and sequence length far exceed those of A2C. Similar results are shown in Figure 10 for the Industry scenario. These observations demonstrate A-iALP’s advantages in convergence speed and stability, leading to greater long-term gains for users.

7 Conclusion and Future Work

We addressed the challenges of offline and online distribution shift and the lack of data exploration in RL based recommender systems, which hinders online deployment of RL based systems. Our investigation focused on utilizing LLMs to pre-train the RL policy, enhancing both initial user gains and long-term effectiveness in recommendation scenarios. By distilling user preferences from LLMs, we created an offline user feedback to train our RL policy, which was then adapted online using our adaptive (A-iALP) methods. Our experiments on three simulated environments demonstrated that A-iALP enhances the quality of initial recommendations while maintaining

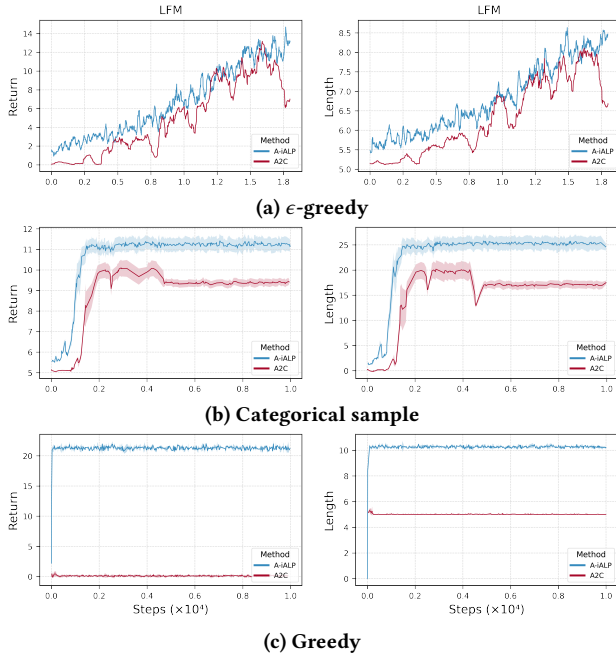


Figure 11: Performance comparison between A-iALP and the baseline under various exploration strategies on LFM.

stability over time. Future research directions include expanding the pre-training dataset with more diverse user interactions and integrating advanced reward mechanisms to further optimize user satisfaction and engagement.

References

- [1] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. *arXiv preprint arXiv:2305.00447* (2023).
- [2] Jin Chen, Zheng Liu, Xu Huang, Chenwang Wu, Qi Liu, Gangwei Jiang, Yuanhao Pu, Yuxuan Lei, Xiaolong Chen, Xingmei Wang, et al. 2023. When large language models meet personalization: Perspectives of challenges and opportunities. *arXiv preprint arXiv:2307.16376* (2023).
- [3] Yingpeng Du, Di Luo, Rui Yan, Hongzhi Liu, Yang Song, Hengshu Zhu, and Jie Zhang. 2023. Enhancing job recommendation through llm-based generative adversarial networks. *arXiv preprint arXiv:2307.10747* (2023).
- [4] Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, and Qing Li. 2023. Recommender systems in the era of large language models (llms). *arXiv preprint arXiv:2307.02046* (2023).
- [5] Amir massoud Farahmand, Rémi Munos, and Csaba Szepesvári. 2010. Error propagation for Approximate Policy and Value Iteration. In *Proceedings of the 23rd International Conference on Neural Information Processing Systems - Volume 1* (Vancouver, British Columbia, Canada) (*NIPS'10*). Curran Associates Inc., Red Hook, NY, USA, 568–576.
- [6] Junchen Fu, Fajie Yuan, Yu Song, Zheng Yuan, Mingyue Cheng, Shenghui Cheng, Jiaqi Zhang, Jie Wang, and Yunzhu Pan. 2024. Exploring adapter-based transfer learning for recommender systems: Empirical studies and practical insights. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, 208–217.
- [7] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*, 299–315.
- [8] Dalin Guo, Sofia Ira Ktena, Pranay Kumar Myana, Ferenc Huszar, Wenzhe Shi, Alykhan Tejani, Michael Kneier, and Sourav Das. 2020. Deep bayesian bandits: Exploring in online personalized recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems*, 456–461.
- [9] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).
- [10] Zhankui He, Zhouhang Xie, Rahul Jha, Harald Steck, Dawen Liang, Yesu Feng, Bodhisattwa Prasad Majumder, Nathan Kallus, and Julian McAuley. 2023. Large language models as zero-shot conversational recommenders. In *Proceedings of the 32nd ACM international conference on information and knowledge management*, 720–730.
- [11] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2015. Session-based recommendations with recurrent neural networks. *arXiv preprint arXiv:1511.06939* (2015).
- [12] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards universal sequence representation learning for recommender systems. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 585–593.
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).
- [14] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).
- [15] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [16] Vijay Konda and John Tsitsiklis. 1999. Actor-critic algorithms. *Advances in neural information processing systems* 12 (1999).
- [17] Volodymyr Kuleshov and Doina Precup. 2014. Algorithms for multi-armed bandit problems. *arXiv preprint arXiv:1402.6028* (2014).
- [18] Aviral Kumar, Justin Fu, George Tucker, and Sergey Levine. 2019. *Stabilizing off-policy Q-learning via bootstrapping error reduction*. Curran Associates Inc., Red Hook, NY, USA.
- [19] Minae Kwon, Sang Michael Xie, Kalesha Bullard, and Dorsa Sadigh. 2023. Reward Design with Language Models. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=10uNUG15Kl>
- [20] Seunghyun Lee, Younggyo Seo, Kimin Lee, Pieter Abbeel, and Jinwoo Shin. 2021. Offline-to-Online Reinforcement Learning via Balanced Replay and Pessimistic Q-Ensemble. In *Annual Conference on Robot Learning*.
- [21] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).
- [22] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, et al. 2023. How Can Recommender Systems Benefit from Large Language Models: A Survey. *arXiv preprint arXiv:2306.05817* (2023).
- [23] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *Comput. Surveys* 55, 9 (2023), 1–35.
- [24] Shuchang Liu, Qingpeng Cai, Bowen Sun, Yuhao Wang, Ji Jiang, Dong Zheng, Peng Jiang, Kun Gai, Xiangyu Zhao, and Yongfeng Zhang. 2023. Exploration and regularization of the latent action space in recommendation. In *Proceedings of the ACM Web Conference 2023*, 833–844.
- [25] Bonan Min, Hayley Ross, Elmor Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *Comput. Surveys* 56, 2 (2023), 1–40.
- [26] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).
- [27] Omar Moling, Linas Baltrunas, and Francesco Ricci. 2012. Optimal radio channel recommendations with explicit and implicit feedback. In *Proceedings of the sixth ACM conference on Recommender systems*, 75–82.
- [28] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, 188–197.
- [29] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (2022), 27730–27744.
- [30] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21, 1 (2020), 5485–5551.
- [31] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q Tran, Jonah Samost, et al. 2023.

- Recommender Systems with Generative Retrieval. *arXiv preprint arXiv:2305.05065* (2023).
- [32] Zhaochun Ren, Na Huang, Yidan Wang, Pengjie Ren, Jun Ma, Jiahuan Lei, Xinlei Shi, Hengliang Luo, Joemon Jose, and Xin Xin. 2023. Contrastive State Augmentations for Reinforcement Learning-Based Recommender Systems. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 922–931.
 - [33] Shideh Rezaeifar, Robert Dadashi, Nino Vieillard, Léonard Hussenot, Olivier Bachem, Olivier Pietquin, and Matthieu Geist. 2022. Offline Reinforcement Learning as Anti-exploration. In *AAAI Conference on Artificial Intelligence*.
 - [34] Markus Schedl. 2016. The lfm-1b dataset for music retrieval and recommendation. In *Proceedings of the 2016 ACM on international conference on multimedia retrieval*. 103–110.
 - [35] Tobias Schnabel, Adith Swaminathan, Ashudeep Singh, Navin Chandak, and Thorsten Joachims. 2016. Recommendations as treatments: Debiasing learning and evaluation. In *international conference on machine learning*. PMLR, 1670–1679.
 - [36] Younggyo Seo, Lili Chen, Jinwoo Shin, Honglak Lee, Pieter Abbeel, and Kimin Lee. 2021. State entropy maximization with random encoders for efficient exploration. In *International Conference on Machine Learning*. PMLR, 9443–9454.
 - [37] Wentao Shi, Xiangnan He, Yang Zhang, Chongming Gao, Xinyue Li, Jizhi Zhang, Qifan Wang, and Fuli Feng. 2024. Large Language Models are Learnable Planners for Long-Term Recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1893–1903.
 - [38] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.
 - [39] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
 - [40] Jie Wang, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M Jose. 2024. Reinforcement Learning-based Recommender Systems with Large Language Models for State Reward and Action Modeling. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 375–385.
 - [41] Jie Wang, Fajie Yuan, Mingyue Cheng, Joemon M Jose, Chenyun Yu, Beibei Kong, Xiangnan He, Zhijin Wang, Bo Hu, and Zang Li. 2022. Transrec: Learning transferable recommendation from mixture-of-modality feedback. *arXiv preprint arXiv:2206.06190* (2022).
 - [42] Pengfei Wang, Yu Fan, Long Xia, Wayne Xin Zhao, ShaoZhang Niu, and Jimmy Huang. 2020. KERL: A knowledge-guided reinforcement learning model for sequential recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 209–218.
 - [43] Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* 8 (1992), 229–256.
 - [44] Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M Jose. 2020. Self-supervised reinforcement learning for recommender systems. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 931–940.
 - [45] Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M Jose. 2022. Supervised advantage actor-critic for recommender systems. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1186–1196.
 - [46] Xin Xin, Tiago Pimentel, Alexandros Karatzoglou, Pengjie Ren, Konstantina Christakopoulou, and Zhaochun Ren. 2022. Rethinking Reinforcement Learning for Recommendation: A Prompt Perspective. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1347–1357.
 - [47] Yuanqing Yu, Chongming Gao, Jiawei Chen, Heng Tang, Yuefeng Sun, Qian Chen, Weizhi Ma, and Min Zhang. 2024. EasyRL4Rec: A User-Friendly Code Library for Reinforcement Learning Based Recommender Systems. *arXiv preprint arXiv:2402.15164* (2024).
 - [48] Fajie Yuan, Alexandros Karatzoglou, Ioannis Arapakis, Joemon M Jose, and Xiangnan He. 2019. A simple convolutional generative network for next item recommendation. In *Proceedings of the twelfth ACM international conference on web search and data mining*. 582–590.
 - [49] Gangyi Zhang. 2023. User-Centric Conversational Recommendation: Adapting the Need of User with Large Language Models. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1349–1354.
 - [50] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
 - [51] Xiangyu Zhao, Liang Zhang, Zhuoye Ding, Long Xia, Jiliang Tang, and Dawei Yin. 2018. Recommendations with negative feedback via pairwise deep reinforcement learning. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1040–1048.